

Optimizacija Distribuiranog Bojenja Grafa

1. UVOD

U ovom projektu implementirane su **dvije verzije** algoritma za distribuirano bojenje grafa:

- **Baseline verzija** - neoptimizovana implementacija za poređenje
- **Optimizovana verzija** - sa primijenjenim tehnikama za poboljšanje performansi

Obe verzije implementiraju **isti algoritam** za bojenje grafa tako da susjedni čvorovi nikada nemaju istu boju, ali se razlikuju u načinu na koji koriste Spark RDD API.

Cilj: Pokazati koliko se može ubrzati algoritam bez mijenjanja njegove logike.

2. PREGLED OPTIMIZACIJA

Aspekt	Baseline Verzija	Optimizovana Verzija
<i>Agregacija boja susjeda</i>	<i>groupByKey()</i>	<i>aggregateByKey()</i>
<i>Keširanje privremenih RDD-ova</i>	<i>MEMORY_AND_DISK</i>	<i>MEMORY_ONLY_SER</i>
<i>Particionisanje</i>	<i>Default (automatsko)</i>	<i>Prilagođeno (konfigurisano)</i>
<i>Shuffle operacije</i>	<i>Visok broj prenosa</i>	<i>Redukovan broj prenosa</i>

Optimizacije omogućavaju:

- **Brže slanje** informacija o tome koje boje koriste susjedi čvora
- **Manje memorije** za čuvanje informacija o bojenju
- **Optimalnu raspodjelu** čvorova između CPU jezgara

3. OPTIMIZACIJE

3.1. Efikasno prikupljanje boja - *aggregateByKey()*

Svaki čvor, prije nego što odabere svoju boju, mora **pitati sve svoje susjede** koje boje oni koriste. Tek kada sazna koje su boje "zauzete", može odabrati slobodnu boju.

Scenario:

Zamislimo konkretnu situaciju iz našeg grafa:

- Graf: čvor 5 ima 100 susjeda
- Trenutno stanje: svi susjedi su već obojeni
- Zadatak čvora 5: Saznaj boje svojih susjeda
- Problem: svi susjedi imaju istu boju (boja 2)

Pitanje: Kako čvor 5 saznaje koje boje koriste njegovi susjedi?

Baseline Pristup:

Svaki susjed čvora 5 šalje posebnu poruku: "*Ja sam obojen bojom 2*".

Čvor 5 prima 100 poruka sa istom informacijom: "*susjed ima boju 2*".

Posledice:

- **Mrežni saobraćaj:** 100 poruka putuje preko mreže između Spark worker-a
- **Memorija:** Čvor 5 mora čuvati listu: [2, 2, 2, 2, ..., 2] (100 elemenata)
- **Vrijeme:** Slanje i primanje 100 poruka traje

Matematika:

- Svaka poruka je oko 8 bytes (ID čvora + boja)
- Ukupno imamo 800 bytes podataka preko mreže

Optimizovani Pristup:

Ovo ćemo predstaviti kroz tri faze.

Faza 1:

Koristi se lokalno kombinovanje (na svakom Spark worker-u) prije slanja

Recimo da se čvorovi 10-109 nalaze na 3 različita Spark worker-a:

Worker 1 (ima čvorove 10-42)

Priprema poruke za čvor 5:

Čvor 10 → boja 2

Čvor 11 → boja 2

...

Čvor 42 → boja 2

Kombinuje preko mreže:

Umjesto 30 poruka "boja 2"

Šalje jednu poruku: {2}

Worker 2 (ima čvorove 43-75)

Kombinuje: {2}

Worker 3 (ima čvorove 76-109)

Kombinuje: {2}

Faza 2:

Prenos preko mreže.

Umjesto 100 pojedinačnih poruka, šalju se samo 3 seta:

Worker 1 → Čvoru 5: {2}

Worker 2 → Čvoru 5: {2}

Worker 3 → Čvoru 5: {2}

Faza 3:

Čvor 5 prima 3 seta i kombinuje ih: $\{2\} \cup \{2\} \cup \{2\} = \{2\}$

Poređenje:

Za čvor 5 sa 100 susjeda (svi boja 2):

Metrika	Baseline	Optimizovano	Poboljšanje
Broj poruka	100	3	33x manje
Veličina podataka	800 bytes	60 bytes	13x manje
Tip podataka	Lista [2,2,2,...]	Set {2}	uklanjanje duplikata
Memorija	800 bytes	20 bytes	40x manje

Zašto je ovo važno za bojenje grafa?

Algoritam bojenja radi iterativno.

Svaka iteracija šalje poruke što znači da se optimizacija multiplikuje kroz sve iteracije.

To za posledicu ima da što graf ima veći broj čvorova, to su beneficije optimizacije sve veće.

Dodatno Set automatski uklanja duplikate.

Matematička analiza

Scenario: Graf sa N čvorova, svaki čvor ima prosječan stepen D

- N = broj čvorova u grafu
- D = prosječan stepen čvora (broj susjeda)
- K = broj boja (obično $K \ll D$)

Metrika	<i>groupByKey()</i>	<i>aggregateByKey()</i>	Poboljšanje
Broj poruka	$N \times D$	$N \times \min(D, K)$	D/K manje
Mrežni saobraćaj	$O(N \times D)$	$O(N \times K)$	D/K manje
Memorija po čvoru	$O(D)$ (lista)	$O(K)$ (set)	D/K manje

Teoretsko Objašnjenje

MapReduce paradigma omogućava dvije vrste kombinovanja:

- Map-side combining** - kombinovanje prije shuffle-a (na izvoru)
- Reduce-side combining** - kombinovanje poslije shuffle-a (na destinaciji)

Ključna razlika:

- groupByKey()* **ne može** raditi *map-side combining* jer ne zna kako kombinovati vrijednosti
- aggregateByKey()* **može** jer mu dajemo funkciju za kombinovanje:
 $((set, color) \rightarrow set.add(color))$

3.2. Kompaktno čuvanje grafa

Tokom algoritma, moramo čuvati trenutno stanje grafa u memoriji:

- Koji čvorovi su već obojeni?
- Koji čvorovi još nisu obojeni?
- Koje boje koriste susjedi?

Problem:

Ako graf ima 10,000 čvorova, kako efikasno čuvati sve te informacije u RAM-u?

Šta je keširanje (persistence) u Spark-u?

Spark RDD-ovi su **lazy**. To znači da se ne računaju odmah, već tek kada se pozove akcija poput: `.count()`, `.collect()`...

Problem:

Ako se koristi isti RDD više puta, Spark će ga preračunavati svaki put.

Rješenje:

`.persist()` - kaže Sparku: "Sačuvaj ovaj RDD u memoriji, koristiću ga ponovo."

Spark Storage Levels

Spark nudi različite načine čuvanja RDD-ova:

Storage Level	Gdje se čuva?	Format	Šta se dešava ako nema RAM-a?
MEMORY_ONLY	RAM	Java objekti (deserijalizovano)	Recompute
MEMORY_ONLY_SER	RAM	Byte nizovi (serijalizovano)	Recompute
MEMORY_AND_DISK	RAM + Disk	Java objekti	Spill na disk
MEMORY_AND_DISK_SER	RAM + Disk	Byte nizovi	Spill na disk

Šta kada nema mjesta?

Recompute - briše sve i računa ponovo.

Spill - sklanja na disk da sačuva.

Baseline Pristup:

Koristi se: `MEMORY_AND_DISK`

To znači da imamo:

- Java objekte u memoriji
- Serijalizovane objekte na disku

Karakteristike

pozitivno:

- **brz pristup** - objekti su već deserijalizovani

negativno:

- **veliki memory footprint** - Java objekti su "teški"
- **Garbage Collection problemi** - mnogo objekata u heap-u
- **Disk I/O overhead** - ako nema RAM-a, spill na disk je spor

Optimizovani Pristup:

Koristi se: MEMORY_ONLY_SER

To znači da imamo:

- Serijalizovane byte nizove

Karakteristike

pozitivno:

- **mala memorija** - 2-5x manje od Java objekata
- **manje GC pauza** - manje objekata u heap-u
- **miše RDD-ova u RAM-u** - efikasnije iskorišćenje memorije

negativno:

- **minimalan CPU overhead** - deserijalizacija pri čitanju (ali brza)
- **recompute ako nema RAM-a** - ne koristi disk, već preračunava

Kada Koristiti Koju Strategiju?

Koristiti MEMORY_AND_DISK kada:

- RDD se koristi **mnogo puta** (10+ pristupa)
- Ima **dovoljno RAM-a**
- **CPU** je ograničavajući faktor (ne želimo deserijalizaciju)
- RDD je **veći od RAM-a** (potreban disk backup)

Koristiti MEMORY_ONLY_SER kada:

- RDD se koristi **2-3 puta** (privremeno keširanje)
- **RAM je ograničavajući faktor** (malo memorije)
- RDD je **manji od RAM-a** (recompute je brz)
- Želimo **više RDD-ova u memoriji istovremeno**

Strategija tokom algoritma bojenja

Algoritam bojenja koristi VIŠE privremenih RDD-ova:

- **Glavni graf** (*nodesRDD*) → MEMORY_AND_DISK (koristi se mnogo)
- **Privremeni podaci** (*uncoloredRDD*) → MEMORY_ONLY_SER (štedi RAM)

3.3. Custom Partitionisanje - optimizacija distribucije čvorova

Cilj: Postizanje maksimalne paralelnosti i izbjegavanje "uskih grla" (*data skew*) kroz ravnomjernu distribuciju čvorova grafova između *worker* čvorova.

Šta su particije u Spark-u?

Particija je osnovna logička jedinica podataka.

Spark dijeli graf na particije (grupe čvorova), pri čemu se svaka particija obrađuje kao zaseban zadatak na jednom CPU jezgri u jednom trenutku.

Analiza problema pri izboru broja particija

Problem 1: premalo particija

Primjer: graf od 100,000 čvorova podijeljen na 8 particija (12,500 čvorova po particiji).

Posljedica: loša granularnost.

Efekat: ako jedna particija sadrži kompleksnije veze (teži podaci), jedno jezgro će ostati opterećeno dugo nakon što ostalih 7 završe posao. Resursi ostaju neiskorišteni dok se čeka najsporiji zadatak.

Problem 2: previše particija

Primjer: mali graf od 100 čvorova podijeljen na 32 particije (3 čvora po particiji).

Posljedica: preveliki administrativni trošak (overhead).

Efekat: Spark troši više vremena na kreiranje zadataka, upravljanje mrežnom komunikacijom i koordinaciju particija nego na stvarnu obradu čvorova.

Optimalna podjela

Spark koristi `defaultParallelism()` koji je obično:

`defaultParallelism` = broj CPU jezgara u clusteru

Za maksimalne performanse koristi se pravilo palca:

optimalan broj particija = (2 do 4) x broj CPU jezgara

Ovaj faktor omogućava dinamičko balansiranje opterećenja. Umjesto jednog velikog zadatka, jezgra dobijaju više manjih particija. Jezgra koja brže završe svoj posao ne čekaju besposleno, već odmah preuzimaju sljedeće dostupne particije. Time se eliminiše "prazan hod" i neutrališe uticaj sporih čvorova, a istovremeno se izbjegava sistemsko zagušenje prevelikim brojem sitnih zadataka.

Praktična upotreba u projektu

U optimizovanoj varijanti postoji mogućnost da se eksplicitno podesi broj particija.

Koristi se CLI opcija:

`--partitions 32`

4. BENCHMARK REZULTATI

Rezultati testiranja

Tabela: Prosječna vremena izvršavanja (10 pokretanja)

Test #	Broj Čvorova	Max Stepen	seed	seed	Baseline (ms)	Optimizovano (ms)	Broj poruka
1	500	3	10	10	4512	4467	4389
2	1000	5	1	1	17347	16316	20567
3	1500	8	2	2	71999	62097	230085
4	2000	10	3	3	347865	360605	128609
5	2500	12	4	4	557554	538009	3429395
6	3000	15	5	5	840866	813972	1766176
7	3500	18	6	6	1234935	1167944	1167944
8	4000	20	7	7	1830781	1791153	832014
9	4500	22	8	8	2303447	2275801	1526441
10	5000	50	9	9	2869628	2704746	6521798

Metodologija tesiranja

Sprovedena su dva testna ciklusa od po 10 mjerenja - jedan za osnovnu (baseline) i jedan za optimizovanu varijantu koda.

Kako bi se osigurala vjerodostojnost rezultata, oba scenarija su koristila identične parametre: isti broj čvorova, isti maksimalni stepen i isti seed (čime je garantovano generisanje identične strukture grafa).

Zaključak tesiranja

Rezultati pokazuju da su ostvarena poboljšanja u ovoj fazi minimalna, prvenstveno zbog relativno malog broja čvorova u testnim grafovima.

Kod manjih skupova podataka, *overhead* Spark okvira često nadmašuje samu uštedu u procesiranju.

Zabilježene su i određene varijacije u performansama, uključujući slučaj gdje je baseline varijanta bila neznatno brža.

Razlike u broju razmijenjenih poruka, posledica su različitih nasumičnih struktura grafova generisanih za svaki pojedinačni par testiranja.

Maksimalni benefiti

Optimizacije daju najbolje rezultate kada se ispune sledeći uslovi:

- Veliki grafovi ($N > 10,000$ čvorova)
- Visok stepen čvorova ($D > 20$)
- Cluster okruženje (više worker čvorova)
- Ograničena memorija (malo RAM-a)

Poboljšanja će biti manje primjetna u sljedećim situacijama:

- Male grafove ($N < 1,000$)
- Lokalno pokretanje (jedan worker)
- Višak resursa (kada graf u potpunosti staje u RAM)